

Experiment – 2

Name: Probal Dhali

UID: 24BAI71013

Branch: CSE (AI ML) CS221

Section/Group: 24AIT_NTPP-3 [B]

Semester: 5th

Date of Performance: 09/07/2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 2.1

1. AIM:

To design and implement a centralized state management system using **Redux Toolkit** for managing posts, platforms, and drafts in a React application.

2. OBJECTIVE:

1. To understand the concept of centralized (global) state management.
2. To implement Redux Toolkit for efficient state handling.
3. To create Redux slices for managing application data.
4. To perform CRUD (Create, Read, Update, Delete) operations using reducers.
5. To connect React components with the Redux Store using React-Redux Hooks.
6. To design a scalable and maintainable application architecture.
7. To reduce prop drilling by using a centralized state.
8. To improve application performance through structured state management.

3. SOFTWARE REQUIREMENTS:

1. Intel Core i3/i5 Processor or higher, 4 GB RAM (Minimum)
2. Ubuntu / Windows 10 or later
3. [Node.js](#), npm, [React.js](#)
4. Redux Toolkit, React-Redux
5. Visual Studio Code, Google Chrome / Mozilla Firefox

4. THEORY

Modern React applications often contain many components that need to share the same data. Passing data from parent to child components repeatedly (known as **Prop Drilling**) makes applications difficult to manage and maintain.

Redux is a predictable state management library that stores all application data inside a single centralized **Store**. Every component can access the required data directly from the store without unnecessary prop passing.

Redux Toolkit (RTK) is the official, recommended way of writing Redux applications. It reduces boilerplate code by providing utilities such as:

- `configureStore()`
- `createSlice()`
- `createAsyncThunk()`

Redux Toolkit improves scalability, readability, maintainability, and performance of React applications.

5. IMPLEMENTATION/PROCEDURE

- Install Node.js and React.
- Create a React project.

- Install Redux Toolkit and React-Redux.
- npm install @reduxjs/toolkit react-redux
- Create the Redux Store.
- Create Redux Slice for Posts.
- Define Initial State.
- Create Reducers for CRUD operations.
- Wrap the React application with the Provider.
- Connect components using useSelector() and useDispatch().
- Perform Create, Update and Delete operations.
- Run the application.
- Verify the output.

6. CODE

```
import { useSelector, useDispatch } from "react-redux";
import { addPost } from "./postsSlice";
```

```
function App(){
  const dispatch = useDispatch();
  const posts = useSelector(state => state.posts.posts);
  return(
    <div>
      <h2>Redux Toolkit Demo</h2>
      <button
        onClick={()=>
          dispatch(addPost({
            id:1,
            title:"Redux Toolkit Experiment"
          }))
        }
      >
      Add Post
    </button>
    <ul>
      {
        posts.map(post=>
          <li key={post.id}>
            {post.title}
          </li>
        )
      }
    </ul>
  </div>
  );
}
```

```
export default App
```

7. OUTPUT

The screenshot displays a web application titled "Redux Toolkit - Post Manager". It features a "Create New Post" section at the top with a text input field labeled "Enter post title", a dropdown menu currently set to "Facebook", and a blue "Add Post" button. Below this is a "Posts" section containing two post cards. The first card is for "Probal Dhali", showing it is on the "Facebook" platform and in "Draft" status, with "Toggle Status" and "Delete" buttons. The second card is for "Chandigarh University", also on the "LinkedIn" platform and in "Draft" status, with similar "Toggle Status" and "Delete" buttons.

Fig 2.2: Output of Redux Toolkit Application displaying centralized state management using Redux Store and React-Redux Hooks.

The application successfully implements centralized state management using **Redux Toolkit**. Users can create new posts, select a social media platform, view all posts, update the post status (Draft/Published), and delete posts. All post data is managed through a centralized Redux Store, reducing prop drilling and improving application scalability.

8. LEARNING OUTCOME

- Understood the concept of centralized state management.
- Learned how Redux Toolkit simplifies Redux development.
- Created and configured a Redux Store.
- Implemented Redux Slices and Reducers.
- Performed CRUD operations using Redux actions.
- Connected React components with Redux using `useSelector()` and `useDispatch()`.
- Reduced prop drilling through centralized state management.
- Developed a scalable and maintainable React application using Redux Toolkit

EXPERIMENT – 2.2

1. AIM:

To optimize state access and improve application performance using **memoized selectors** and efficient rendering strategies in a React application with Redux Toolkit.

2. OBJECTIVE:

1. To understand the concept of derived state.
2. To implement memoized selectors using **Reselect (createSelector)**.
3. To reduce unnecessary component re-renders.
4. To improve application performance using optimized state access.
5. To compute derived data without duplicating state.
6. To integrate selectors into React components.
7. To use React optimization techniques such as **React.memo** and **useMemo**.
8. To design a scalable and high-performance frontend architecture.

3. SOFTWARE REQUIREMENTS:

1. Intel Core i3/i5 Processor or higher
2. 4 GB RAM (Minimum), 10 GB Free Disk Space
3. Ubuntu / Windows 10 or later
4. [Node.js](#), npm, [React.js](#)
5. Redux Toolkit, React-Redux
6. Reselect (or built-in [createSelector](#))
7. Visual Studio Code, Google Chrome / Mozilla Firefox

4. THEORY

As React applications grow in size, components frequently access shared data from the Redux Store. Without optimization, selectors recalculate values every time the state changes, even if the relevant data remains unchanged. This results in unnecessary computations and excessive component re-rendering, reducing application performance.

Selectors are functions that retrieve specific data from the Redux Store. Memoized selectors, created using `createSelector()` from Redux Toolkit or Reselect, cache computed values and only recompute them when their input data changes.

Derived state refers to values calculated from existing state rather than stored separately. Examples include filtered posts, completed tasks, grouped records, or statistical summaries. Using memoized selectors avoids redundant calculations while keeping the Redux Store normalized.

React also provides optimization techniques such as `React.memo()`, which prevents unnecessary re-rendering of components when props remain unchanged, and `useMemo()`, which caches expensive calculations inside functional components.

By combining Redux Toolkit selectors with React optimization techniques, applications become more scalable, responsive, and efficient.

5. IMPLEMENTATION/PROCEDURE

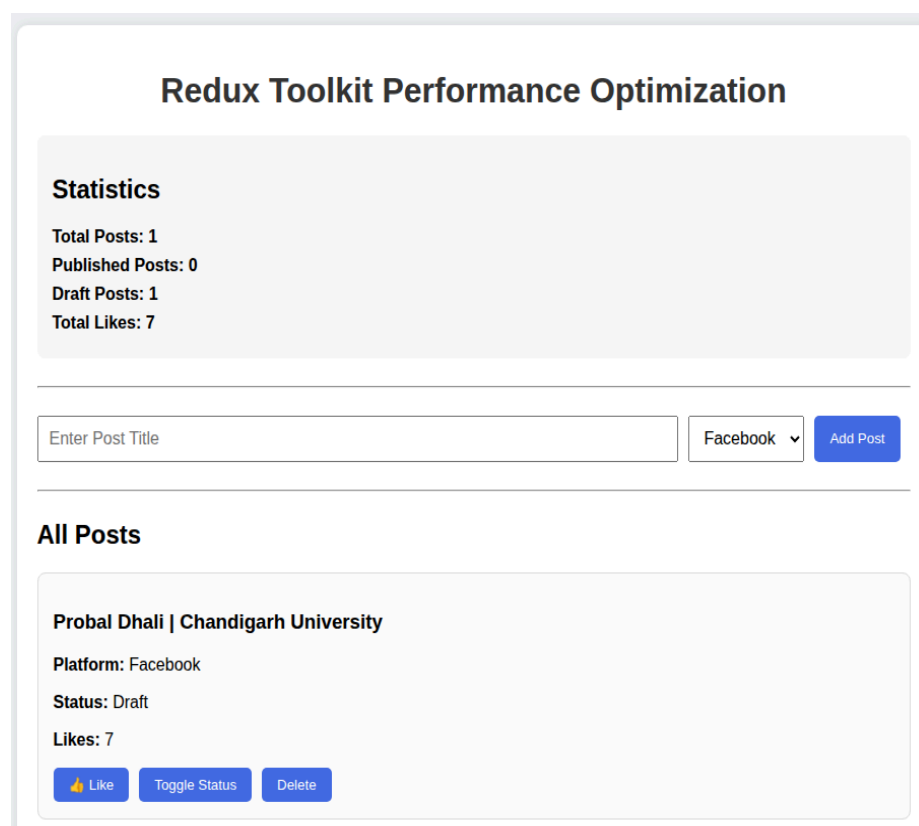
1. Create a React project.
2. Install Redux Toolkit and React-Redux.
3. Install Reselect (optional if using Redux Toolkit's built-in `createSelector`).
`npm install @reduxjs/toolkit react-redux reselect`
4. Configure the Redux Store.
5. Create a Posts Slice with sample post data.

6. Define basic selectors for accessing posts.
7. Create memoized selectors using `createSelector()`.
8. Compute derived data such as published posts.
9. Connect React components using `useSelector()`.
10. Optimize components using `React.memo()`.
11. Run the application.
12. Verify optimized rendering and filtered output.

6. CODE

```
import React from "react";
import { useSelector } from "react-redux";
import { selectPublishedPosts } from "../selectors";
const App = () => {
  const publishedPosts = useSelector(selectPublishedPosts);
  return (
    <div>
      <h2>Published Posts</h2>
      <ul>
        {publishedPosts.map((post) => (
          <li key={post.id}>
            {post.title}
          </li>
        ))}
      </ul>
    </div>
  );
};
export default React.memo(App);
;
```

7. OUTPUT



- Fig 2.1: Output of React application displaying filtered published posts using memoized selectors (`createSelector`) with optimized rendering through `React.memo()`.

The application successfully demonstrates performance optimization using **memoized selectors** (**createSelector**) and efficient state management. It displays real-time statistics such as **Total Posts, Published Posts, Draft Posts, and Total Likes**, while allowing users to add posts, like posts, change status, and delete posts. Optimized selectors reduce unnecessary computations and component re-renders, resulting in better application performance.

8. LEARNING OUTCOME

- Understood the concept of derived state in Redux.
- Learned how memoized selectors improve application performance.
- Implemented **createSelector()** for efficient state access.
- Reduced unnecessary computations using memoization.
- Minimized component re-rendering using **React.memo()**.
- Retrieved filtered data efficiently from the Redux Store.
- Improved scalability and maintainability of React applications.
- Gained practical knowledge of performance optimization techniques in Redux Toolkit.